MKSSS's

Cummins College of Engineering for Women, Pune

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Third Year B. Tech (Computer Engineering)

MINI PROJECT REPORT

On

"BioScan: A Reference-Aware DNA Barcode Identification System with Out-of-Reference Detection for Fish Environmental DNA"

Submitted by

Student Name: Shreya Jadhav

U. No.: _____

Class / Division: _____

Roll No.: _____

Under the Guidance of

Guide Name: _____

Department of Computer Engineering

Academic Year: 2026–27

TABLE OF CONTENTS

Sr. No.	Contents	Page No.
1	Introduction	3
2	Motivation	4
3	Literature Survey	4
4	Dataset Used	6
5	Flow Diagram / Block Diagram	8
6	Methods Used / Technology Stack	9
7	Model Development	10
8	Version Control	12
9	Experiment Tracking	13
10	AutoML / MLOps	14
11	Deployment	14
12	Results and Discussions	20
13	Future Scope	26
14	Conclusion	26
15	References	27

List of figures and tables

Figures and tables are numbered sequentially. Figure captions appear below the figure and table captions above the table, as required by the report template.

1. Introduction

1.1 Background

Every animal sheds traces of its DNA into the water, soil and air around it through skin, mucus, faeces and gametes. Environmental DNA (eDNA) is the genetic material that can be collected from such samples without ever capturing the organism. A litre of seawater can be filtered, the captured DNA amplified with primers aimed at a short, informative gene region (a DNA barcode), and the resulting sequences compared with a library of reference barcodes to learn which species are present. For fishes, a widely used target is a hypervariable region of the mitochondrial 12S ribosomal RNA gene, amplified with the MiFish family of primers [1].

1.2 Importance of the problem

eDNA surveys allow fisheries managers, conservation agencies and ecologists to monitor biodiversity at a scale and frequency that trawling or visual surveys cannot match. They are used to detect rare or endangered species, to follow invasive species and to describe the composition of marine communities [10]. The reliability of these surveys rests on a single step: assigning a taxonomic name to each DNA sequence.

1.3 Existing challenges

- **Incomplete reference libraries.** No reference library covers every species. A sequence from a species that is missing from the library still has a closest match, and a naive pipeline reports that match as if it were an identification.
- **Closed-set classifiers over-claim.** A supervised classifier must output one of the labels it was trained on. Its probabilities sum to one over the known labels, so an unfamiliar sequence can receive a confident but wrong species name.
- **Very few examples per species.** Reference libraries usually hold one or two barcodes per species. This makes species-level machine learning and its honest evaluation difficult.
- **Repeated records inflate scores.** Survey tables repeat each unique sequence across samples and primer replicates. Splitting such rows at random puts identical sequences in both the training and test sets and produces misleadingly high accuracy.

1.4 Role of AI/ML and the proposed solution

This project presents BioScan, a reference-aware identification system for fish 12S barcodes. Instead of asking only “which label is most likely?”, BioScan also asks “is this sequence actually supported by the reference library?”. It combines three kinds of evidence: (i)

hierarchical machine-learning classifiers that predict the taxonomy at class, order, family, genus and species level from k-mer features; (ii) an exact edit-distance search against the reference barcodes; and (iii) a Distance Veto, a rule that declares a sequence out-of-reference when its nearest reference is too far away, whatever the classifier says. A Streamlit dashboard presents the result, the evidence and the measured reliability of the method.

1.5 Expected outcome

The expected outcome is a working, tested system that (a) names a taxon only as deep as the evidence supports, (b) refuses to name sequences that fall outside its library, (c) reports its own accuracy using leakage-free, held-out evaluation, and (d) clearly states what the small reference library cannot support, in particular species-level generalisation.

2. Motivation

The project was chosen because misidentification in eDNA studies has practical consequences. A wrongly reported species can trigger an invasive-species alarm, hide the presence of a protected species or distort a diversity estimate. The most common failure is not that a classifier is inaccurate on species it knows, but that it is silently overconfident on species it has never seen.

Existing approaches leave a gap. A BLAST search [2] returns a best hit but leaves the decision of how similar is similar enough to the analyst. A trained classifier [3] returns a label and a score, but the score is relative to the known classes only. A single accuracy figure computed on duplicated rows hides how little independent data there is. An automated, transparent system that combines both kinds of evidence, abstains when evidence is weak and reports honest evaluation addresses all three problems.

Expected benefits are: (i) fewer silent misidentifications; (ii) a clear, graded answer (species, family, order, or none) instead of a forced species name; (iii) fast batch screening of many sequences from a FASTA file; and (iv) a documented evaluation that others can reproduce. Classical machine-learning models (k-nearest neighbours, random forest, gradient boosting) were selected because the dataset is small and tabular, where they are appropriate and fast, and because an edit-distance search is exact and interpretable at this library size.

3. Literature Survey

The literature relevant to BioScan covers three areas: the laboratory marker used for fish eDNA, computational methods for assigning taxonomy to sequences, and methods that recognise inputs from classes that were never seen in training. Table 1 summarises five representative works. Curated reference libraries such as the Barcode of Life Data System [5] and NCBI GenBank provide the sequences against which all of these methods compare.

Table 1. Literature survey of related work

Sr. No.	Author(s)	Year	Title / Method	Dataset	Technique Used	Major Findings	Limitation
1	M. Miya et al. [1]	2015	MiFish universal primers for fish eDNA metabarcoding	Fish mitochondrial genomes and field water samples	PCR primer design on 12S rRNA, high-throughput sequencing, database matching	A short 12S region can be amplified across diverse fishes and detects hundreds of species in water samples	Identification depends on the reference database; species absent from it cannot be named
2	S. F. Altschul et al. [2]	1990	BLAST: Basic Local Alignment Search Tool	General nucleotide and protein databases	Heuristic local alignment with statistical scoring	Fast, sensitive similarity search that became the standard for sequence identification	Reports a best hit even when the true taxon is absent; thresholds are left to the user
3	Q. Wang et al. [3]	2007	Naïve Bayesian classifier for rRNA taxonomy (RDP)	16S rRNA sequences with curated taxonomy	Naive Bayes on 8-mer words with bootstrap confidence	Fast, accurate assignment to genus level with a confidence estimate	Designed for bacterial 16S; assigns only within its training taxonomy
4	B. J. Callahan et al. [4]	2016	DADA2: sample inference from Illumina amplicon data	Mock and environmental amplicon datasets	Error model and denoising to exact amplicon sequence variants	Resolves single-base differences and reduces spurious sequences	Pre-processing only; does not decide whether a taxon is in the library
5	W. J. Scheirer et al. [11]	2013	Toward open set recognition	Computer-vision benchmark datasets	Formalises open-space risk; one-vs-set machine	A recogniser should reject inputs from classes unseen in training	Studied mainly on image data, not biological sequences

Commonly used techniques. Alignment-based search [2] is exact and widely trusted but delivers a similarity score and not a decision. k-mer composition classifiers [3] are fast and give a bootstrap confidence but are closed-set. Denoising tools [4] improve the input but do not address coverage. Open-set recognition [11], [12] provides the right conceptual framework, in which a system must reject what it has not seen, but it has seen little use in barcode identification.

Identified gap. To our knowledge, no commonly used barcode pipeline couples hierarchical learning with an explicit, evaluated rejection rule and reports its performance on held-out, de-duplicated data with the limits of the library stated. BioScan addresses this gap with a transparent distance-based veto, a rank-by-rank answer that stops where the evidence stops, and an evaluation that separates taxonomy accuracy from unknown-sequence detection.

4. Dataset Used

4.1 Source and size

The data come from the 2019 New Jersey Bureau of Marine Fisheries Ocean Trawl eDNA survey, deposited in the NCBI BioProject PRJNA638560 [15]. The raw table (hackathon_dna_taxonomy_merged.csv) contains 13,736 rows and 59 columns. The marker is the mitochondrial 12S rRNA V5 region amplified with MiFish-style primers. The SHA-256 checksum of the raw file was recorded and re-verified after processing to show that it was never modified.

4.2 Duplicate structure

The 13,736 rows are not independent examples. They contain only 101 unique DNA sequences, each repeated exactly 136 times (101 sequences × 68 sampling events × 2 primer replicates). Treating the rows as independent would place the same sequence in training and test data, so all modelling and evaluation use the 101 unique sequences, and the repeated rows are used only to build occurrence records for the survey map.

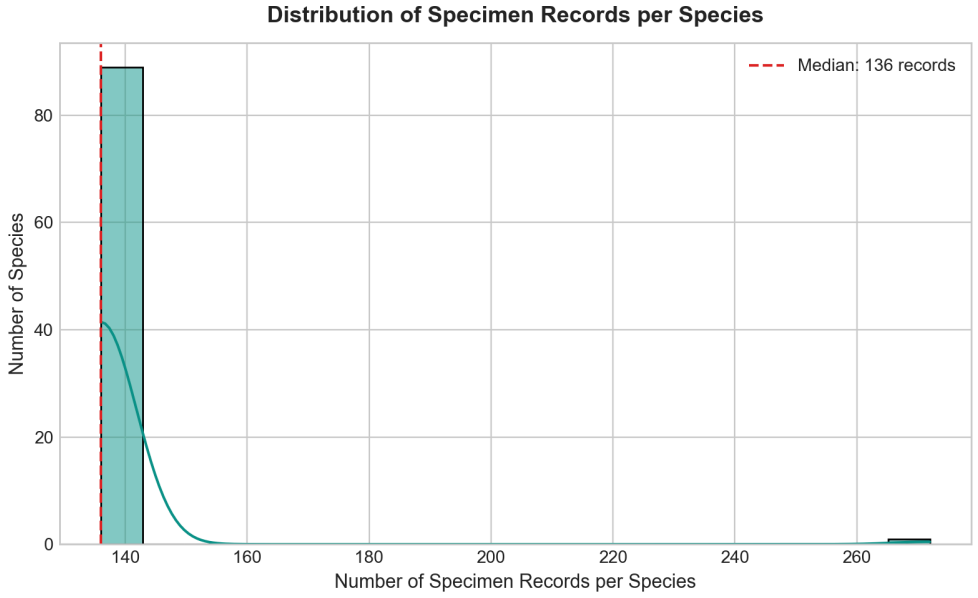


Figure 1. Raw rows per species: each sequence is repeated 136 times, so the raw table holds only 101 independent barcodes.

4.3 Reference library characteristics

Table 2. Summary of the reference library

Property	Value
Unique reference barcodes	101
Sequence length	87 to 112 bp (mean 105.7, median 106; 86.1% are 106 bp)
Alphabet	Only A, C, G, T (no ambiguity codes)
GC content	46.5% ± 2.6%
Taxonomy	4 classes, 30 orders, 66 families, 87 genera, 90 named species
Species labels	91 sequences labelled; 10 sequences lack a species name
Class balance	Teleostei 82, Elasmobranchii 17, Petromyzonti 1, Chondrostei 1
Singleton species	89 of 90 species have exactly one sequence
Survey metadata	36 stations, 1,604 de-duplicated occurrence records, 98 of 101 barcodes detected

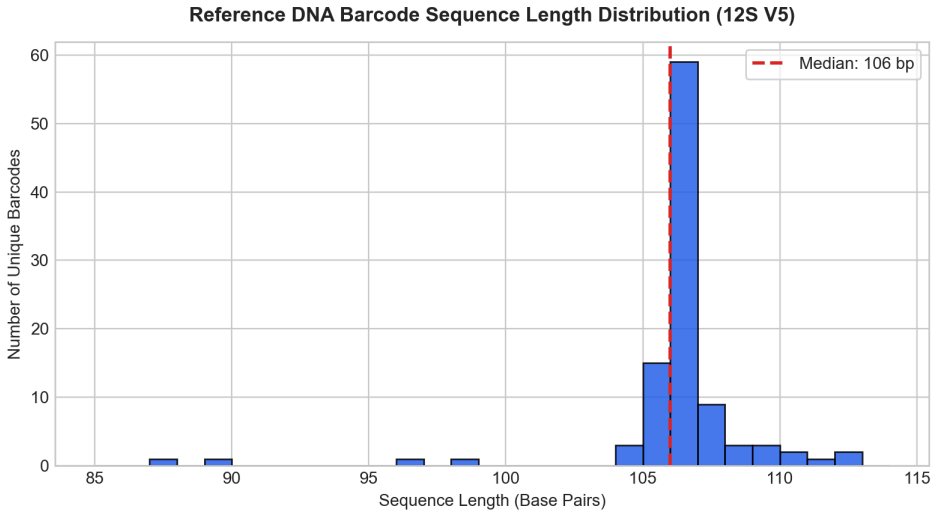


Figure 2. Distribution of reference sequence lengths (87 to 112 bp).

4.4 Feature description

Machine-learning features were computed only from the nucleotide string. Station, date, depth, coordinates and read counts were excluded from the features to prevent leakage and are used only afterwards, for biodiversity context. There is no missing value in the sequence column.

Table 3. Feature description

Feature	Data Type	Description	Role
sequence_length	Integer	Length of the amplicon in base pairs	Input
A/C/G/T count	Integer	Number of each nucleotide	Input

A/C/G/T frequency	Float	Count divided by length	Input
GC_content, AT_content	Float	Fraction of G+C and of A+T bases	Input
unique_base / k2 / k3 count	Integer	Number of distinct bases, dinucleotides and trinucleotides observed	Input
shannon_entropy	Float	Shannon information entropy [14] of the base composition (bits)	Input
max / min base frequency	Float	Frequency of the most and least frequent base	Input
k-mer frequencies (k = 2, 3)	Float	16 dinucleotide and 64 trinucleotide relative frequencies	Input
class, order, family, genus, species	Categorical	Taxonomic labels of the reference barcode	Target

4.5 Preprocessing and data split

Preprocessing consisted of upper-casing, whitespace removal, strict validation against the alphabet {A, C, G, T}, de-duplication by normalised sequence, and verification that every duplicate carried identical taxonomy. No outlier removal or imputation was needed. Because 89 of 90 species appear once, conventional k-fold splitting is not possible. Evaluation therefore uses leave-one-sequence-out cross-validation over unique sequences, and singleton classes are reported separately and not scored (Section 7.5).

5. Flow Diagram / Block Diagram

Figure 3 shows the overall workflow of BioScan. A sequence passes through quality control, then follows two parallel paths: a feature-based hierarchical classifier and an exact similarity search. The evidence engine combines both, and the Distance Veto makes the final accept or reject decision before the result and its biodiversity context are shown in the dashboard.

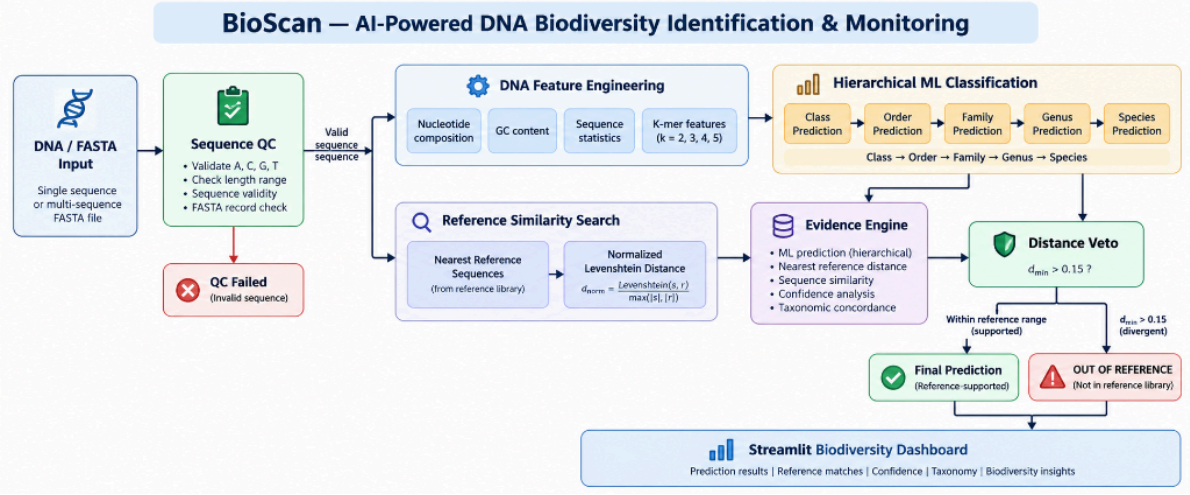


Figure 3. Block diagram of the BioScan identification pipeline.

- **Sequence QC.** Uppercases and cleans the input, accepts only A, C, G and T, records length and GC content, and warns about unusual lengths. Records with other characters are rejected without stopping a batch.
- **Feature engineering.** Computes composition, complexity and k-mer frequency features, using the same code at training and prediction time.
- **Hierarchical classification.** Separate models predict class, order, family, genus and species. Each rank is restricted to children of the parent rank already chosen, which keeps the lineage consistent.
- **Reference similarity search.** Computes the normalised Levenshtein distance to all 101 reference barcodes and returns the nearest references with their taxonomy.
- **Evidence engine.** Combines classifier probability, margin and entropy, nearest distance, the distance gap to the second-nearest reference and agreement between the classifier and the nearest reference.
- **Distance Veto.** If the nearest distance exceeds 0.15 and the match is not exact, the status is out_of_reference regardless of the classifier. No taxonomy or geography is inherited from the nearest reference in that case.
- **Dashboard.** Shows the verdict, distance gauge, what the evidence supports, similarity table, diagnostics, survey map and a separate evaluation page.

6. Methods Used / Technology Stack

6.1 Programming language

Python 3.14 was used for all components: data processing, modelling, the prediction engine, the dashboard and the tests.

6.2 Libraries and frameworks

Table 4. Technologies and libraries actually used

Technology / Library	Purpose and main functions used
pandas 2.3.3	Loading and grouping tabular data: read_csv, groupby, merge, drop_duplicates, DataFrame styling for the dashboard
NumPy 2.2.6	Numerical arrays, sampling and distance arithmetic
scikit-learn 1.7.2 [8]	KNeighborsClassifier, RandomForestClassifier, StandardScaler, LabelEncoder, precision_recall_fscore_support, balanced_accuracy_score, confusion_matrix, roc_auc_score
XGBoost 3.2.0 [6]	XGBClassifier (gradient-boosted trees, multi:softprob objective)
joblib 1.6.0	Saving and loading trained model artefacts
SciPy	Supporting numerical routines

Matplotlib 3.10.9, Seaborn 0.13.2	Static figures used in the analysis reports and in this report
Plotly 7.1.0 [17]	Interactive charts in the dashboard: gauges, heatmaps, maps, accuracy-versus-coverage curves
Streamlit 1.66 [16]	Web dashboard: pages, inputs, file upload, caching of the loaded models
pytest 9.1.1	Automated tests (172 tests)
Custom code	Levenshtein dynamic-programming edit distance, k-mer encoder, evidence and Distance Veto logic, evaluation framework

6.3 Development environment

Development and experiments were carried out on a macOS laptop (CPU only; no GPU or cloud platform was used) with Python 3.14.4 in a virtual environment, Visual Studio Code as the editor, and Git/GitHub for version control. XGBoost requires the OpenMP runtime (libomp) on macOS.

7. Model Development

7.1 Data preprocessing

Raw sequences were normalised (strip, upper-case), validated against {A, C, G, T}, de-duplicated and linked to their taxonomy. The 136 duplicates of each sequence were collapsed into a single reference record after checking that their taxonomic labels agreed. The raw file was never altered.

7.2 Exploratory data analysis

Analysis established the central facts used in the rest of the project: 101 unique sequences, near-uniform length (86% are 106 bp), a base composition of about 30.5% A, 28.5% C, 23.0% T and 18.0% G, a heavy class imbalance (82 Teleostei, 17 Elasmobranchii and two singleton classes) and a singleton structure at species level (Table 2 and Figures 1 and 2). GC and AT content are perfectly collinear, and most k-mer features are correlated with composition.

7.3 Feature engineering / selection

Four feature representations were compared: 17 basic composition and complexity features; 16 dinucleotide frequencies (k = 2); 64 trinucleotide frequencies (k = 3); and basic features combined with k = 2. Larger k-mer sets (k = 4 and 5, with 256 and 1,024 features) were built and compared in the baseline experiments but dropped from the hierarchical comparison, because with 101 samples the feature-to-sample ratio becomes too high. The feature set used by the production model at each rank was chosen from the leave-one-out results: k = 3 for

class, order, family and genus, and k = 2 for species. Features are standardised with a scaler fitted only on the training fold.

7.4 Model selection

Table 5. Models used and their key parameters

Model	Purpose	Important Parameters
K-Nearest Neighbours [13]	Instance-based baseline; classifies by the closest training sequence in feature space	k = 1, Manhattan distance, standardised features
Random Forest [7]	Bagged decision trees; robust with little tuning	100 trees, no depth limit, random_state = 42
XGBoost [6]	Gradient-boosted trees; tests whether boosting helps	100 estimators, max_depth 3, learning rate 0.1, subsample 0.8, colsample_bytree 0.8, softprob, seed 42
Levenshtein search [9] (non-learned)	Exact normalised edit distance to the reference library; drives the Distance Veto	distance = edits / max(length); close ≤ 0.03, veto > 0.15

The deployed model at each rank is the best performer by leave-one-out evaluation: Random Forest for class and order, and k-nearest neighbours for family, genus and species. The species classifier is not allowed to name a species on its own; a species is reported only when the nearest reference lies within the close-match distance.

7.5 Model training and validation

Training and validation follow three principles.

- **Leave-one-sequence-out cross-validation.** Each unique sequence is held out once and predicted by a model trained on all the others, with the scaler refitted in every fold.
- **Singleton handling.** A sequence whose class has no other member cannot be predicted by a closed-set model. Such sequences are counted and reported but excluded from the metrics. For example, 14 of 30 orders and 49 of 66 families are singletons.
- **Minimum evidence rule.** A rank is scored only if at least 20 sequences and two classes can be held out. Genus (19 evaluable sequences) and species (2) therefore show “insufficient data” and no score.

Hyperparameters were explored with small grids: $k \in \{1, 3, 5\}$ for the nearest-neighbour model, 100 or 300 trees with depth unlimited or 10 for the random forest, and 72 XGBoost runs (8 settings of estimators 100/200, depth 3/5 and learning rate 0.05/0.1, across 9 feature sets). Because the data cannot support a separate tuning set, the grids are small and the headline comparison uses fixed, documented settings. Class imbalance is handled by

reporting macro-averaged metrics, which weight every class equally, rather than by resampling, which is not meaningful with singleton classes.

7.6 Model evaluation

Taxonomy classifiers are evaluated with macro precision, macro recall and macro F1 (each class counted equally), weighted F1 (classes weighted by size), balanced accuracy (mean per-class recall), top-1 accuracy and the confusion matrix. Unknown-sequence detection is evaluated separately with out-of-reference as the positive class: precision, recall, F1, specificity, false-positive rate (known sequences wrongly rejected), false-negative rate (unknown sequences wrongly accepted), coverage and abstention rate. Four categories of test sequence are kept separate, because they do not represent the same kind of unknown (Table 5).

Table 6. Categories used to evaluate out-of-reference detection

Test category	How it is built	True label
Known reference (exact)	Each reference queried against the library	Known (sanity check only)
Altered known	Each reference with 1, 2 or 5 random base substitutions	Known by construction
Held-out genus	All sequences of a genus removed from the library, then queried	Out of reference at genus level
External non-fish	106-bp windows of NCBI mammal, bird, reptile and amphibian 12S records	Out of reference

References, genera and external organisms are split 50/50 into a calibration half and a test half with a fixed seed; a detection threshold chosen on the calibration half is reported on the test half. All figures reported in Section 12 use the test half.

8. Version Control

The source code, documentation, reports and trained model artefacts were managed with Git and hosted on GitHub. A .gitignore excludes the virtual environment and temporary files; a README documents installation, usage, validation results and limitations; and the raw dataset checksum is recorded so that its integrity can be verified.

Table 7. Repository summary

Parameter	Details
Version Control Tool	Git
Repository Platform	GitHub
Repository Name	Vedika-k187/BioScan (https://github.com/Vedika-k187/BioScan)
Main Branch	main

Number of Contributors	2 (Vedika Kurhade and Shreya Jadhav)
Major Versions / Commits	8 commits, 14 to 15 October 2026

Table 8. Principal commits

Date	Author	Commit message
2026-10-14	Vedika	Initial commit: BioScan AI-powered DNA biodiversity identification and monitoring system (Phases 1 to 10)
2026-10-14	Vedika	Add Phase 10 final release milestone report
2026-10-14	Vedika	Complete UI polish and pre-GitHub release audit
2026-10-14	Vedika	README updates and architecture diagram
2026-10-15	Shreya Jadhav	Fix species assignment, add unknown-detection validation, dark single-report UI
2026-10-15	Shreya Jadhav	Add evaluation dashboard, rank-resolution block, batch fixes and UI polish

[Insert a screenshot of the GitHub repository page here before submission.]

9. Experiment Tracking

No dedicated experiment-tracking platform (such as MLflow or Weights & Biases) was used. Experiments were recorded manually and reproducibly: every run writes its parameters and metrics to CSV or JSON files in the reports directory (for example phase3_model_results.csv, phase5_taxonomy_results.csv and evaluation_results.json), and each experiment has a written report. The best model per rank was selected from these tables. Table 8 lists the main experiments.

Table 9. Summary of experiments

Experiment	Model	Parameters	Accuracy	F1 Score	Remarks
Exp-01	KNN, Random Forest (species)	9 feature sets; k ∈ {1,3,5}; 100/300 trees	LOOCV 0.022	n/a	Hits the structural ceiling of 2/91: 89 species have one sequence
Exp-02	XGBoost (species)	72 runs: 8 settings × 9 feature sets	LOOCV 0.000	n/a	Boosting cannot learn classes with one example
Exp-03	KNN, RF, XGBoost per rank	60 runs: 5 ranks × 4 feature sets × 3 models	LOOCV total: class 0.98, order 0.64, family 0.45, genus 0.16	n/a	Accuracy falls with depth; singletons count as errors in these totals

Exp-04	Leakage-safe evaluation (final)	Leave-one-sequence-out, unique sequences	Top-1: class 1.00, order 0.76, family 0.87	Macro: class 1.00, order 0.59, family 0.79	Used for all results in Section 12
Exp-05	Distance Veto (out-of-reference)	Close 0.03, veto 0.15; 4 test categories	Recall 35.2%, FPR 0.0%	F1 0.52	AUROC of distance 0.90; 100% of non-fish rejected

10. AutoML / MLOps

10.1 AutoML

AutoML was not used. Model families and hyperparameter grids were chosen manually and are small by design, because the dataset has only 101 independent sequences and an automated search would overfit the validation folds.

10.2 MLOps

A limited set of MLOps practices were implemented; the remaining ones were not.

Table 10. MLOps practices

Practice	Status	Details
Code versioning	Implemented	Git and GitHub
Data versioning / integrity	Partly	Raw file is never modified and its SHA-256 checksum is verified; no data-versioning tool
Model versioning	Partly	Model artefacts (joblib) stored in the repository; no model registry
Experiment tracking	Manual	CSV and JSON result files (Section 9)
Automated testing	Implemented	172 pytest tests covering data, features, models, prediction, evaluation and the Distance Veto
Reproducibility	Implemented	Fixed random seeds, documented splits, regenerable evaluation script
CI/CD	Not implemented	Tests are run manually
Model monitoring	Not implemented	No production traffic to monitor

11. Deployment

BioScan was developed as a local application. The trained models are saved with joblib and loaded once by the prediction engine; a Streamlit dashboard calls the engine directly, so no separate API server is used. The workflow followed was: train and select models → save

artefacts → build the prediction engine → integrate it into the dashboard → test (pytest and manual page checks) → run locally. Public cloud deployment has not been performed.

Table 11. Deployment components

Component	Technology
Application Framework	Streamlit
API Framework	None (Python module interface: predict_sequence, predict_batch, predict_fasta)
Containerization	Not used
Cloud Platform	Not used
Deployment Method	Local execution with: streamlit run app/app.py

On a development laptop the models and reference data load in about one second and a single sequence is classified in under 0.1 seconds, with a peak memory use of about 250 MB. Figures 4 to 10 show the deployed application.

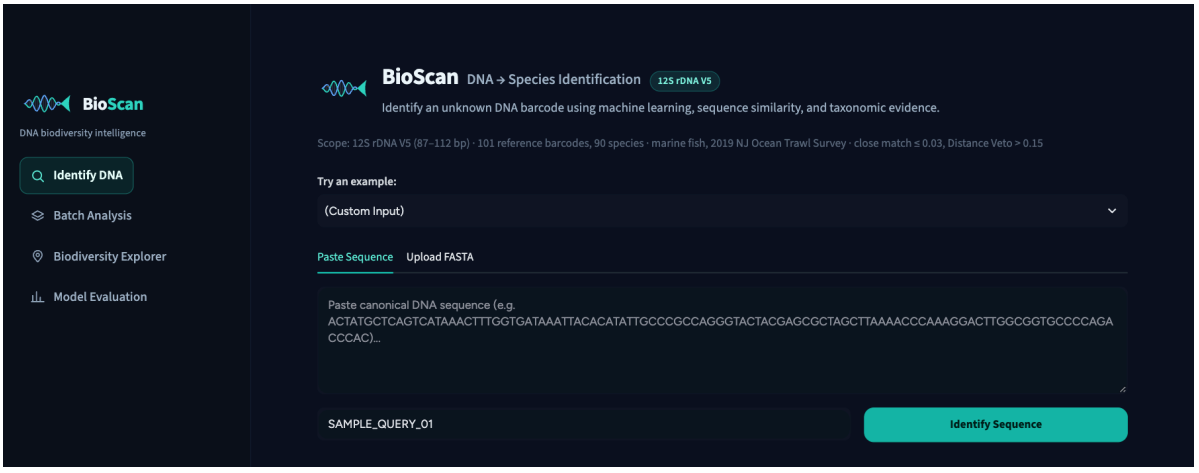


Figure 4. Identify page with sidebar navigation, library summary and result legend.



Figure 5. Result for a diverged sequence: low-confidence verdict, distance gauge and the rank the evidence supports (family).



Figure 6. Reference similarity table and model diagnostics for the same sequence.

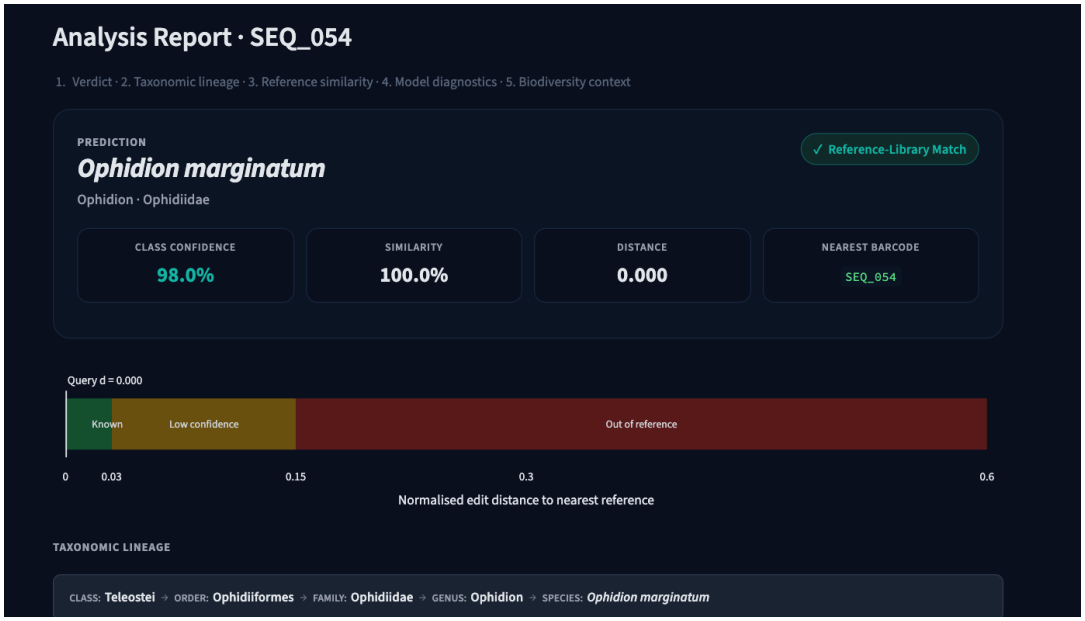


Figure 7. Result for a known reference sequence: exact match with the species named.

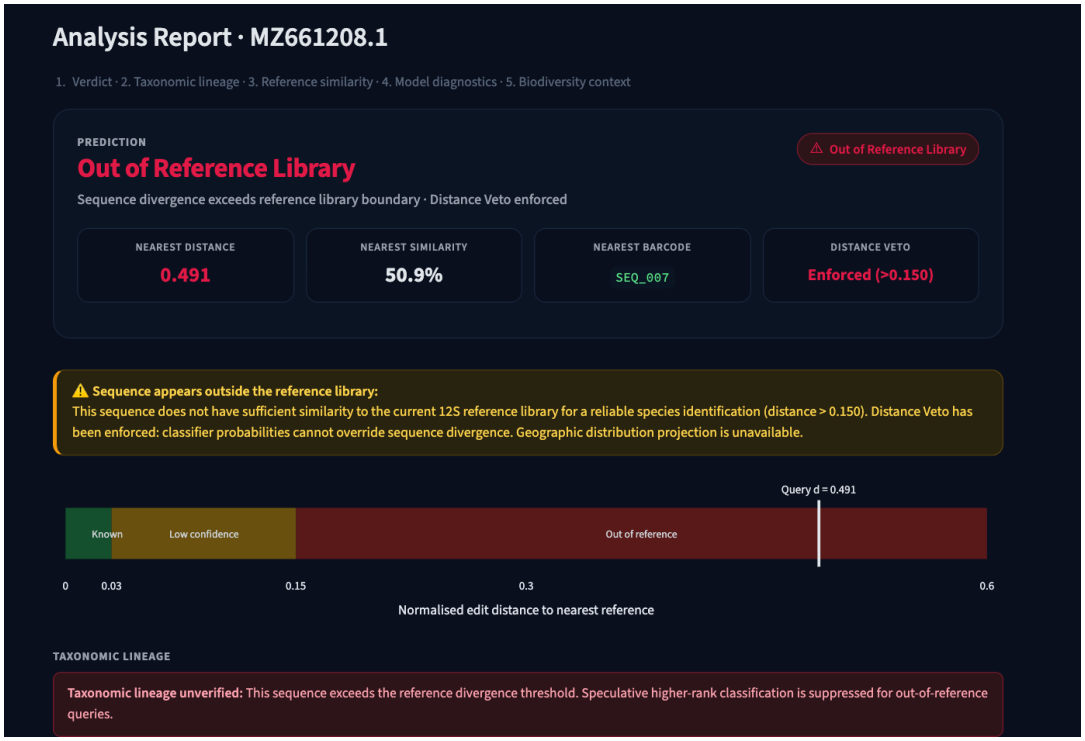


Figure 8. Result for a non-fish sequence: the Distance Veto rejects it and withholds all taxonomy.

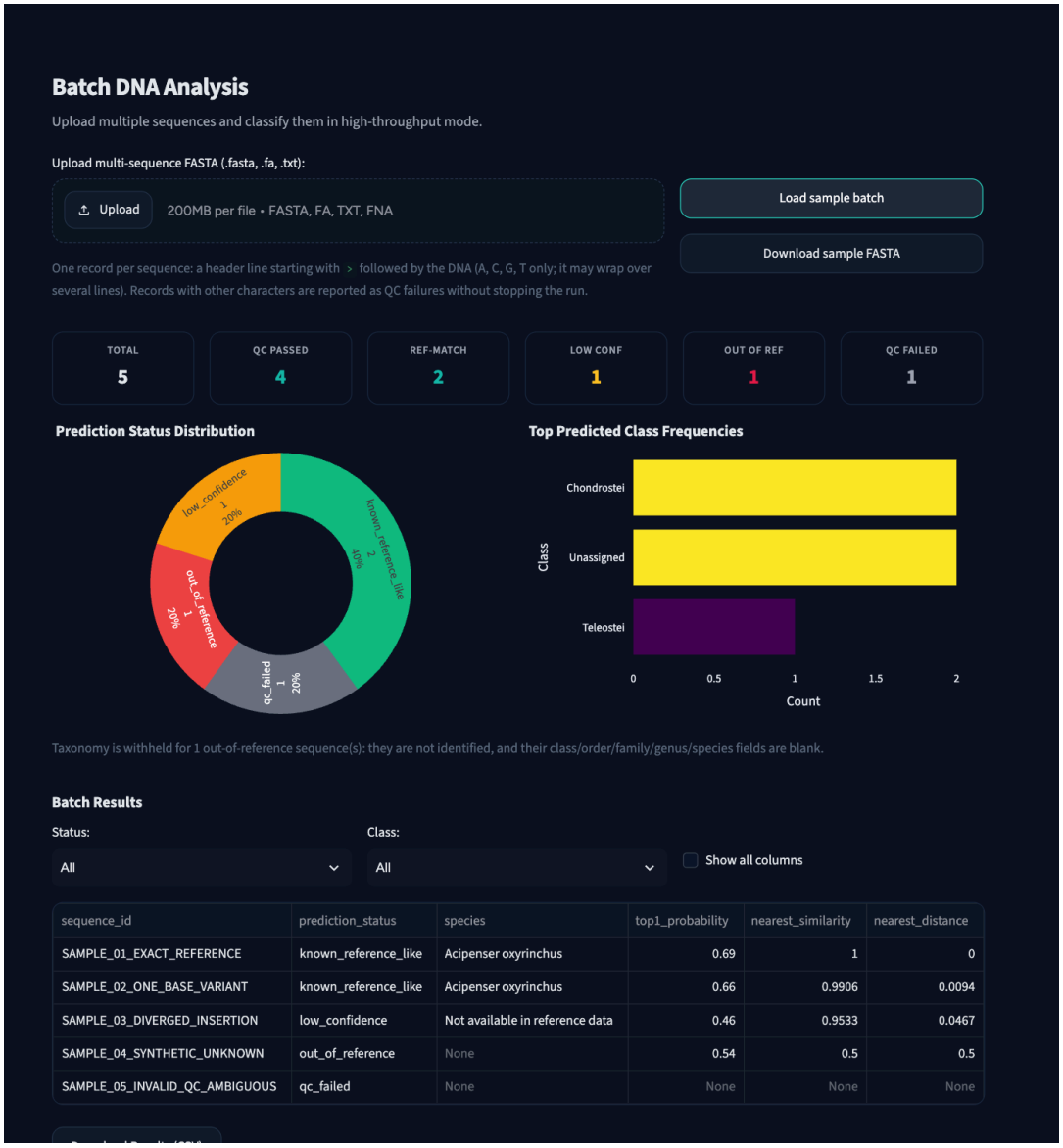


Figure 9. Batch analysis of a five-record FASTA file, showing per-record status and CSV export.

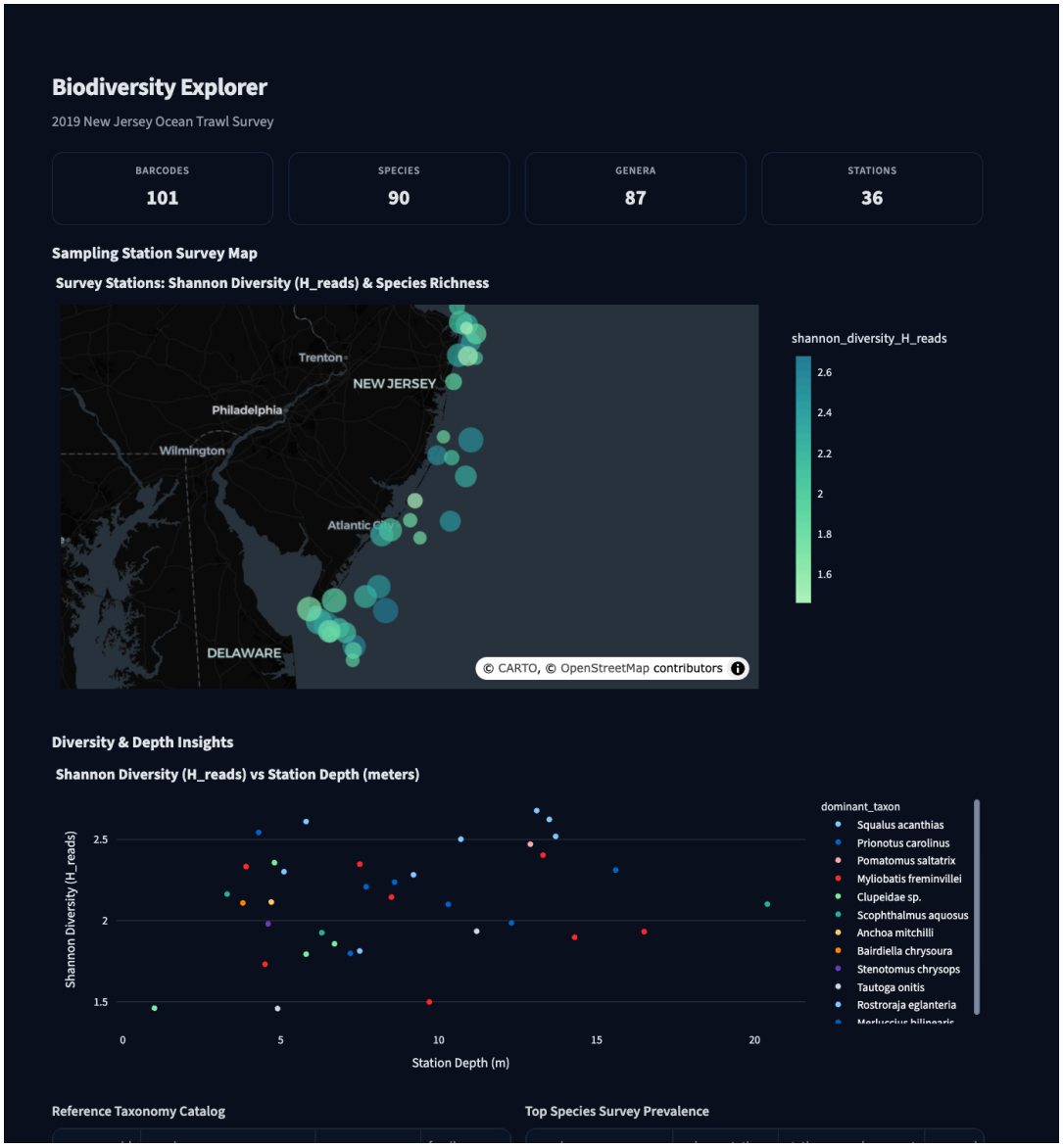


Figure 10. Biodiversity explorer showing survey stations and taxonomic composition.

12. Results and Discussions

12.1 Taxonomy classification

Table 11 reports held-out performance for the three model families at each rank on unique sequences. Genus and species are omitted because too few sequences can be held out to give a meaningful score. Class-level results come from only two testable classes (Teleostei and Elasmobranchii), so they describe an almost binary task.

Table 12. Held-out taxonomy performance (leave-one-sequence-out). * = model used in the application

Rank (test n, classes)	Model	Macro P	Macro R	Macro F1	Weighted F1	Balanced acc.	Top-1 acc.
Class (99, 2 of 4)	KNN	1.000	0.994	0.997	0.995	0.994	0.990

	Random Forest *	1.000	1.000	1.000	1.000	1.000	1.000
	XGBoost	0.972	0.994	0.983	0.990	0.994	0.990
Order (87, 16 of 30)	KNN	0.642	0.647	0.630	0.720	0.647	0.736
	Random Forest *	0.593	0.601	0.586	0.705	0.601	0.759
	XGBoost	0.360	0.423	0.383	0.616	0.423	0.667
Family (52, 17 of 66)	KNN *	0.804	0.794	0.792	0.864	0.794	0.865
	Random Forest	0.722	0.681	0.685	0.749	0.681	0.769
	XGBoost	0.347	0.402	0.367	0.552	0.402	0.596
Genus (19, 9 classes)	Not scored	–	–	–	–	–	–
Species (2, 1 class)	Not scored	–	–	–	–	–	–

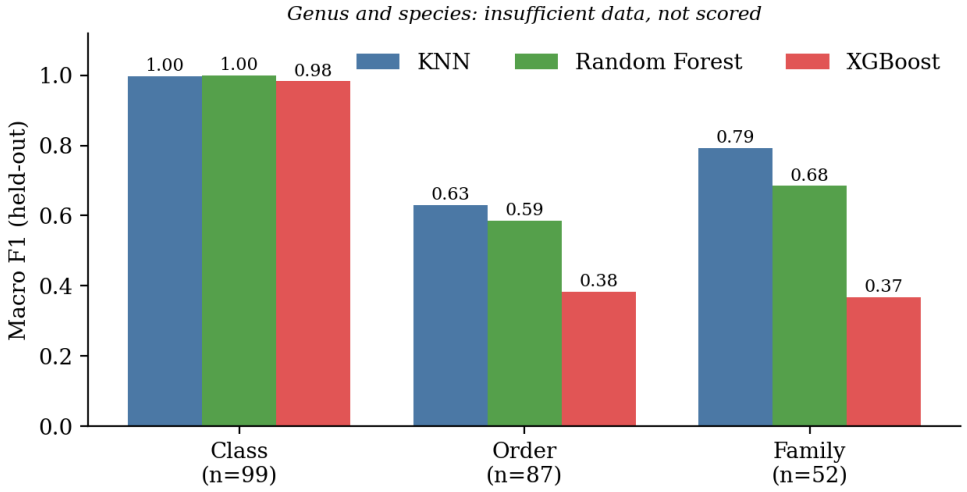


Figure 11. Macro F1 by model and rank on held-out sequences.

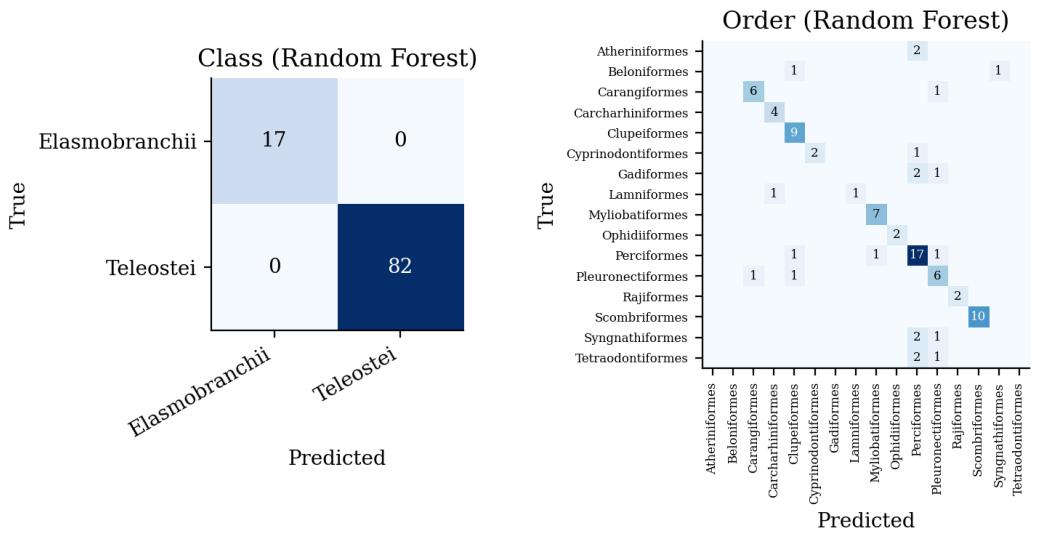


Figure 12. Confusion matrices for class (left) and order (right), Random Forest.

Discussion. Accuracy falls from class to family, which mirrors the data: the number of classes grows and the number of examples per class shrinks. XGBoost is clearly the weakest at order and family (macro F1 0.38 and 0.37), as boosting needs more examples per class than are available. Random forest is best at class and order, and the one-nearest-neighbour model is best at family. At order level a macro F1 of 0.59 against a top-1 accuracy of 0.76 shows that rare orders are missed more often than common ones, which is the reason macro metrics are reported.

An important finding is that a plain nearest-reference search (leave-one-out) labels 100%, 81.6% and 90.4% of sequences correctly at class, order and family, which is comparable to or better than the trained models (class 100%, order 75.9%, family 86.5%). The machine-learning layer therefore adds little beyond class level on this library, and the value of BioScan lies in the evidence combination and the Distance Veto rather than in a better classifier. Figure 13 shows how label accuracy depends on how close the nearest reference is.

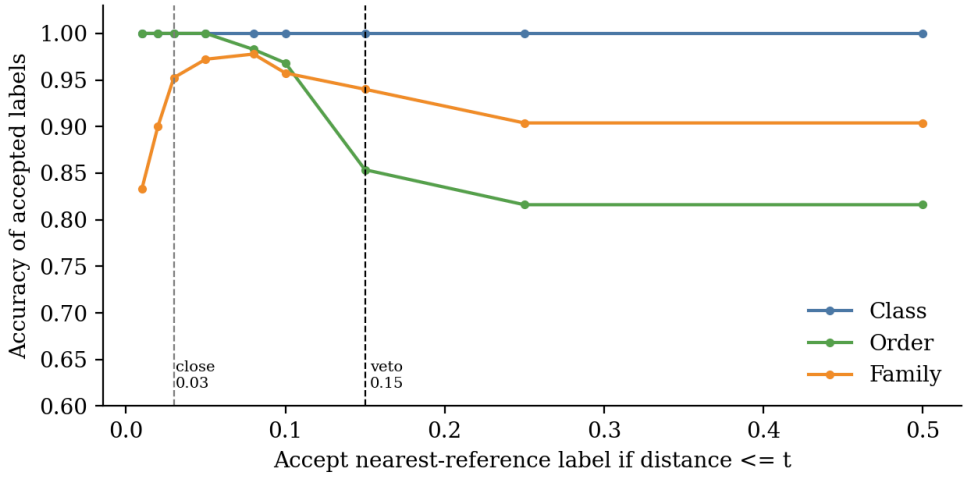


Figure 13. Accuracy of the nearest-reference label as a function of the allowed distance (leave-one-out).

12.2 Out-of-reference detection

Table 12 gives detection performance on the test half: 153 altered known sequences (negatives) and 71 unknown sequences (52 from held-out genera and 19 non-fish). The production rule flags a sequence as out-of-reference only when its distance exceeds 0.15.

Table 13. Out-of-reference detection on the held-out test half (positive = out of reference)

Rule	Precision	Recall	F1	Specificity	False-positive rate	False-negative rate	Coverage
Production veto (flag out of reference)	1.000	0.352	0.521	1.000	0.000	0.648	0.482
Reject or abstain (low confidence counts)	0.560	0.915	0.695	0.667	0.333	0.085	0.482
Threshold 0.05 chosen on calibration half	0.946	0.746	0.835	0.980	0.020	0.254	0.750

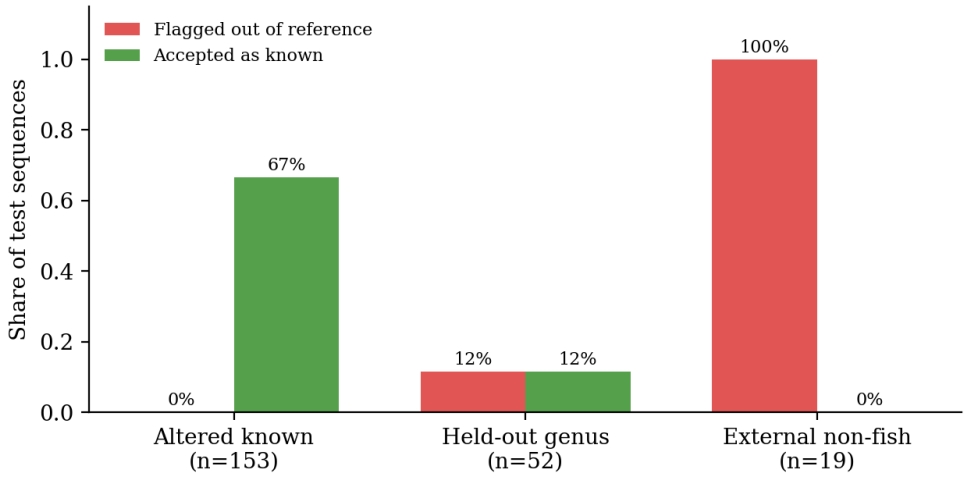


Figure 14. Share of each test category flagged as out of reference (red) or accepted as known (green).

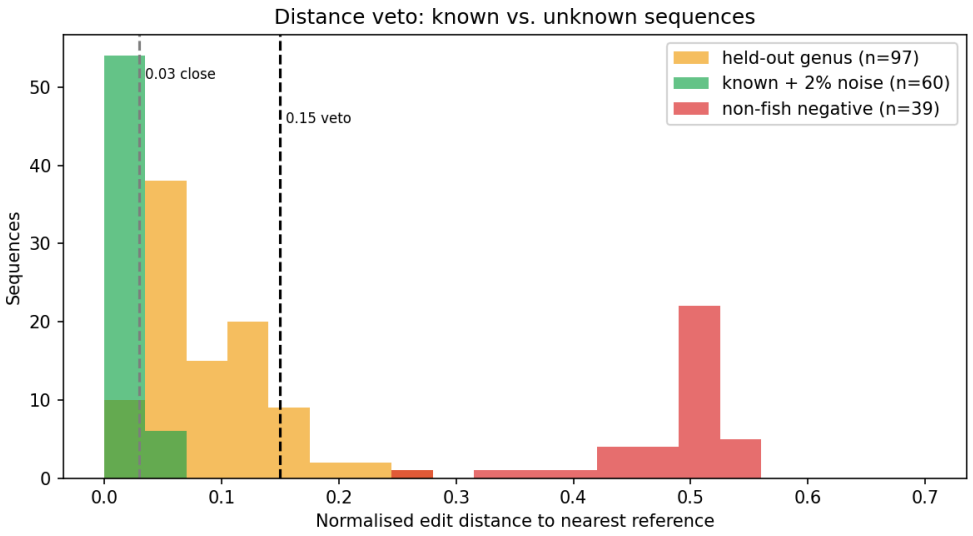


Figure 15. Distance to the nearest reference for altered known (green), held-out genus (orange) and non-fish (red) sequences.

Discussion. The veto is highly specific and precise: no altered known sequence was wrongly rejected, and all 19 non-fish sequences were rejected. Its recall of 35.2% is low because most held-out genera still have a relative in the library and fall between the close-match line (0.03) and the veto (0.15). Those sequences are not accepted wrongly: only 11.5% of held-out genera were accepted as known; the rest were abstained on. Counting abstention as a flag, recall rises to 91.5% at the cost of a 33.3% false-positive rate, because sequences with five base changes (distance about 0.05) are also abstained on. The AUROC of the nearest distance as an unknown score is 0.90.

Table 14. Robustness to substitution errors (test half)

Substitutions	n	Median distance	Max distance	Wrongly rejected	Accepted as known	Abstained
1	51	0.009	0.011	0.0%	100%	0%
2	51	0.019	0.023	0.0%	100%	0%
5	51	0.047	0.058	0.0%	0%	100%

Up to two substitutions in a 100 bp sequence are tolerated and still named; with five substitutions every sequence is moved into the abstention band, which is a conservative behaviour. Insertions, deletions and real sequencing errors were not tested.

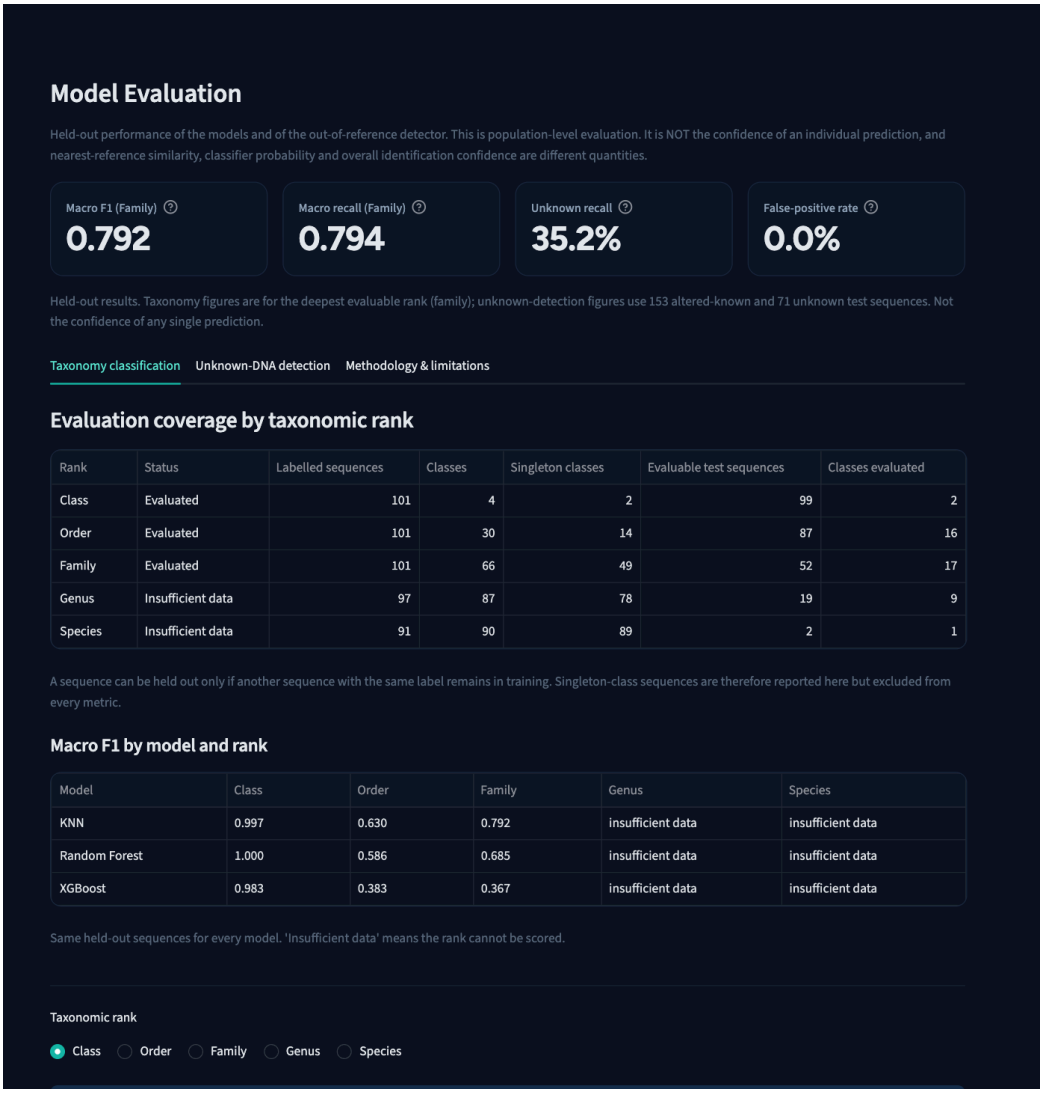


Figure 16. Model Evaluation page of the dashboard: headline cards and coverage by rank.

12.3 Limitations and suitability

- The library has 101 unique sequences, and 89 of 90 species have one. Genus and species generalisation cannot be evaluated, and the species output is reference matching and not a validated classifier.
- Class-level scores come from two testable classes; order and family have a mean of about five and three test sequences per class, so per-class values are unstable.
- The production thresholds (0.03 and 0.15) were originally chosen with these same sequences, so results for them are optimistic; the calibration-half result is the leakage-free estimate.
- Held-out genera often have a close relative in the library. The test shows distance to a relative and not true novelty, and out-of-reference does not prove a new species.
- Non-fish negatives are windows of full-length records and not amplicon reads. No BLAST comparison and no real eDNA sample were evaluated.

Suitability. BioScan is suitable as a proof of concept of the decision logic and as a screening aid for a known, small reference set. It is not suitable as a standalone species identifier for field surveys until the library, the read-level pre-processing and the external validation described in Section 13 are in place.

13. Future Scope

- Expand the reference library with independently sourced, vouchered 12S sequences (several per species) from curated databases, enabling genuine species-level evaluation.
- Benchmark against BLAST or vsearch on the same held-out queries, and compare accepted-identification accuracy and abstention.
- Add a FASTQ-to-ASV front end (primer trimming, denoising, chimera removal) so that real eDNA reads can be analysed directly.
- Re-calibrate the close-match and veto thresholds on larger labelled data, with confidence intervals, and add tolerance for ambiguous bases and reverse-complement reads.
- Support additional markers and taxa (for example COI and 16S) with their own reference libraries.
- Show an alignment view with mismatches in the dashboard for explainability.
- Containerise the application, add continuous integration for the tests, and deploy it on a public cloud service with input-size limits.

14. Conclusion

This project addressed the problem of naming DNA barcodes when the reference library is small and incomplete. BioScan combines hierarchical machine-learning classifiers on k-mer features with an exact edit-distance search and a Distance Veto, so that it names a taxon only as deep as the evidence supports and rejects sequences that fall outside its library. It was built in Python with scikit-learn, XGBoost and Streamlit, and verified with 172 automated tests.

Evaluated on held-out, de-duplicated sequences, the system classifies class, order and family with macro F1 of 1.00, 0.59 and 0.79, rejects all tested non-fish DNA and never wrongly rejects an altered known sequence, while abstaining on most sequences from unseen genera. The evaluation also shows what the small library cannot support: genus and species cannot be validated, and a simple nearest-reference search is as accurate as the trained models above class level. The main practical contribution is therefore a transparent decision framework and an honest evaluation methodology, which is ready to be strengthened by a larger reference library.

15. References

[1] M. Miya, Y. Sato, T. Fukunaga, T. Sado, J. Y. Poulsen, K. Sato, T. Minamoto, S. Yamamoto, H. Yamanaka, H. Araki, M. Kondoh, and W. Iwasaki, “MiFish, a set of universal PCR primers for metabarcoding environmental DNA from fishes: detection of more than 230 subtropical marine species,” Royal Society Open Science, vol. 2, no. 7, p. 150088, 2015.

[2] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, “Basic local alignment search tool,” Journal of Molecular Biology, vol. 215, no. 3, pp. 403–410, 1990.

[3] Q. Wang, G. M. Garrity, J. M. Tiedje, and J. R. Cole, “Naïve Bayesian classifier for rapid assignment of rRNA sequences into the new bacterial taxonomy,” Applied and Environmental Microbiology, vol. 73, no. 16, pp. 5261–5267, 2007.

[4] B. J. Callahan, P. J. McMurdie, M. J. Rosen, A. W. Han, A. J. A. Johnson, and S. P. Holmes, “DADA2: High-resolution sample inference from Illumina amplicon data,” Nature Methods, vol. 13, no. 7, pp. 581–583, 2016.

[5] S. Ratnasingham and P. D. N. Hebert, “BOLD: The Barcode of Life Data System,” Molecular Ecology Notes, vol. 7, no. 3, pp. 355–364, 2007.

[6] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[7] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[8] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[10] K. Deiner, H. M. Bik, E. Mächler, M. Seymour, A. Lacoursière-Roussel, F. Altermatt, S. Creer, I. Bista, D. M. Lodge, N. de Vere, M. E. Pfrender, and L. Bernatchez, “Environmental DNA metabarcoding: Transforming how we survey animal and plant communities,” Molecular Ecology, vol. 26, no. 21, pp. 5872–5895, 2017.

[11] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boult, “Toward open set recognition,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 35, no. 7, pp. 1757–1772, 2013.

[12] C. Geng, S.-J. Huang, and S. Chen, “Recent advances in open set recognition: A survey,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 43, no. 10, pp. 3614–3631, 2021.

[13] T. Cover and P. Hart, “Nearest neighbor pattern classification,” IEEE Transactions on Information Theory, vol. 13, no. 1, pp. 21–27, 1967.

[14] C. E. Shannon, “A mathematical theory of communication,” Bell System Technical Journal, vol. 27, no. 3, pp. 379–423, 1948.

[15] NCBI, “BioProject PRJNA638560: 2019 New Jersey Ocean Trawl Survey environmental DNA,” National Center for Biotechnology Information. [Online]. Available: https://www.ncbi.nlm.nih.gov/bioproject/PRJNA638560. [Accessed: 15-Oct-2026].

[16] Streamlit Inc., “Streamlit Documentation.” [Online]. Available: https://docs.streamlit.io. [Accessed: 15-Oct-2026].

[17] Plotly Technologies Inc., “Plotly Python Graphing Library Documentation.” [Online]. Available: https://plotly.com/python/. [Accessed: 15-Oct-2026].

Appendix A. Running and reproducing the project

The project can be reproduced from the repository with the following steps.

- Create a virtual environment and install the dependencies from requirements.txt (macOS also needs: brew install libomp).
- Run the automated tests with: pytest -q (172 tests).
- Regenerate the evaluation results with: PYTHONPATH=. python src/evaluation/run_model_evaluation.py
- Start the dashboard with: streamlit run app/app.py